

Assignment 6 Technical Report

GUI-Based Multi-Client TCP Chat Application

Student Name: Parth Rawat | Roll No: 0126cy231042

Institution: Tezpur University

1. Objective

The primary objective of this assignment is to design and implement a fully functional graphical desktop client application using Python's tkinter library that interfaces seamlessly with the multi-client TCP server built during Assignment 5. This framework transitions network terminal commands into an event-driven GUI system, ensuring reliable multithreaded non-blocking background network socket messaging inside a Mininet topology environment.

2. GUI Design & Layout

The application frontend architecture is split dynamically into two distinct window workflows: a clean **Authentication Login Window** capturing explicit fields (Username, Server IP address, and Port number), and a comprehensive **Main Chat Window Layout**. The primary interface is composed of an extensive multi-line scrollable chat display area, a live synchronized listbox panel detailing active online participants, dedicated controls to toggle between Global Broadcast or Private message routing, and a clean interface teardown via a Disconnect action button.

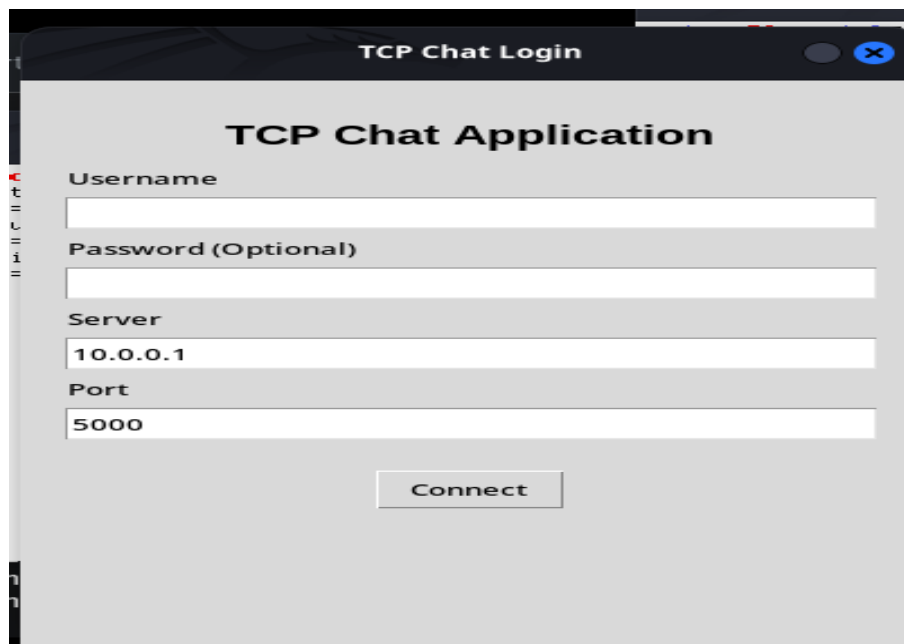


Figure 1: Graphical User Authentication & Socket Login UI Window

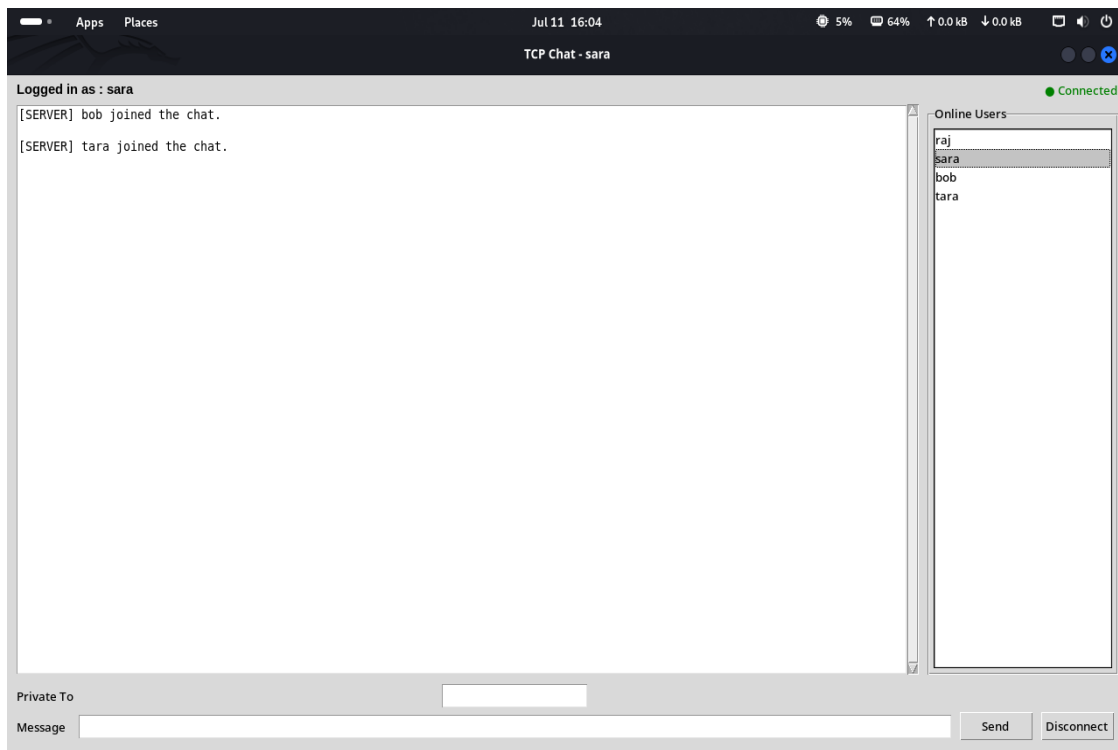


Figure 2: Primary Graphical Multi-Client Interactive Chat Dashboard Window Layout

3. System Architecture

The software pipeline maintains a robust architectural separation between event-driven presentation logic and persistent network socket communication lines. The client uses an asynchronous bridge via a background worker thread: the main UI loop captures user inputs and renders frames while a secondary worker continuously executes blocking network reads (`socket.recv()`), parsing packets into a thread-safe synchronized FIFO queue layer to ensure the interface never encounters blocking pauses during high packet latency.

4. Components Reused from Assignment 5

In compliance with development boundaries, the operational TCP backend engine (`server.py`) was fully reused with complete structural integrity. Preserved backend structures include the multi-client connection thread pools, synchronized resource locks, background state verification routines, persistent CSV-based auditing structures (`chat_history.csv`), server operations reporting metrics, and structural message packet frames for both internal messaging commands and open broadcasts.

5. GUI Design Decisions

To achieve cross-platform compliance inside virtualized Mininet hosts without heavy package weights, standard built-in libraries (`tkinter`, `ttk`, and `scrolledtext`) were strategically deployed. Instead of running concurrent raw operations, a decoupled message queuing mechanism was implemented to route raw network strings safely across thread boundaries, effectively mitigating thread race conditions during rapid state synchronization changes.

6. Functional Topology Testing Results

The integrated application stack was thoroughly verified inside a Mininet environment utilizing a five-node single switch configuration topology (sudo mn --topo single,5) allocating host node h1 as the central TCP server engine and client hosts h2 through h5 executing parallel client instances.

Verification Feature Phase	Observed Operational Performance Summary Metrics
Global Message Broadcasting	Broadcast actions distribute strings instantly to all connected socket instances globally without layout lag.
Targeted Private Messaging	Private message structures evaluate exact username routes and isolates target communications via protocol markers safely.
Online Presence Tracking	Synchronized active user panels reflect live network list changes across all window instances dynamically.
Clean Session Disconnection	Closing actions close sockets gracefully, freeing server descriptors and alerting active peers immediately.

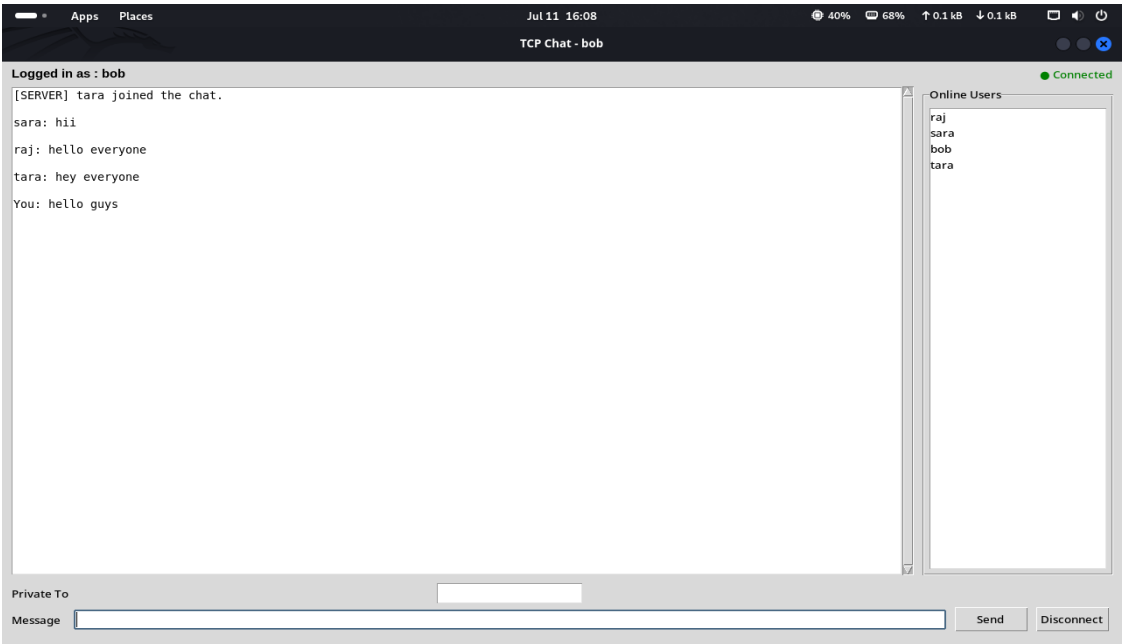


Figure 3: Global Chat Window Broadcasting Execution Trace

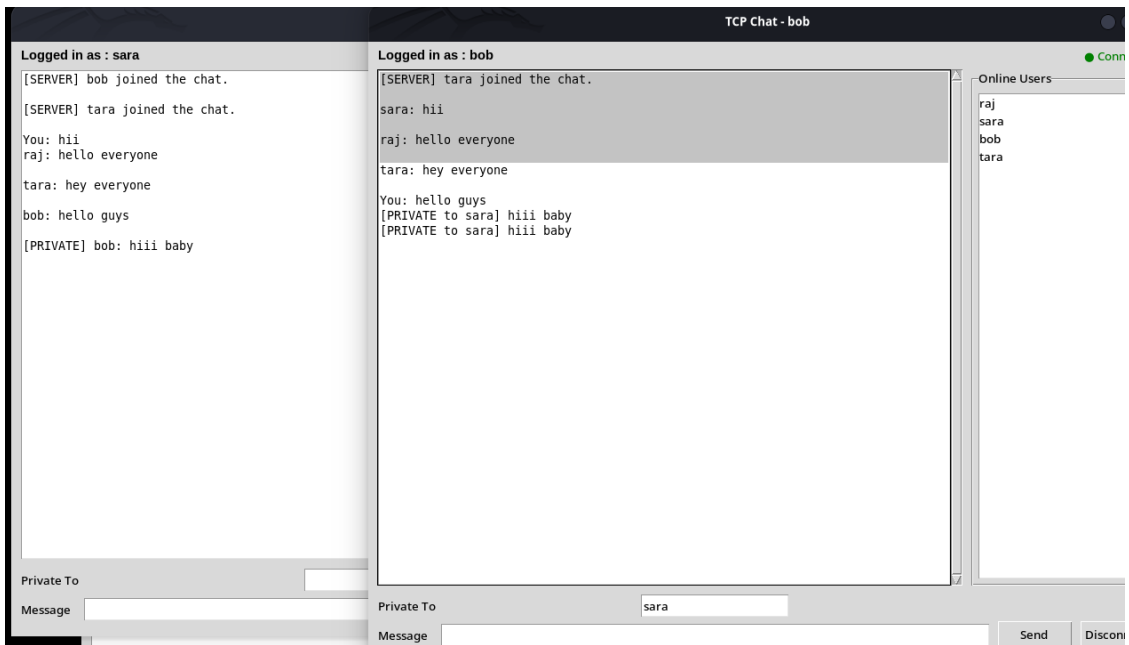


Figure 4: Secure Targeted Private Client-to-Client Chat Isolation Test

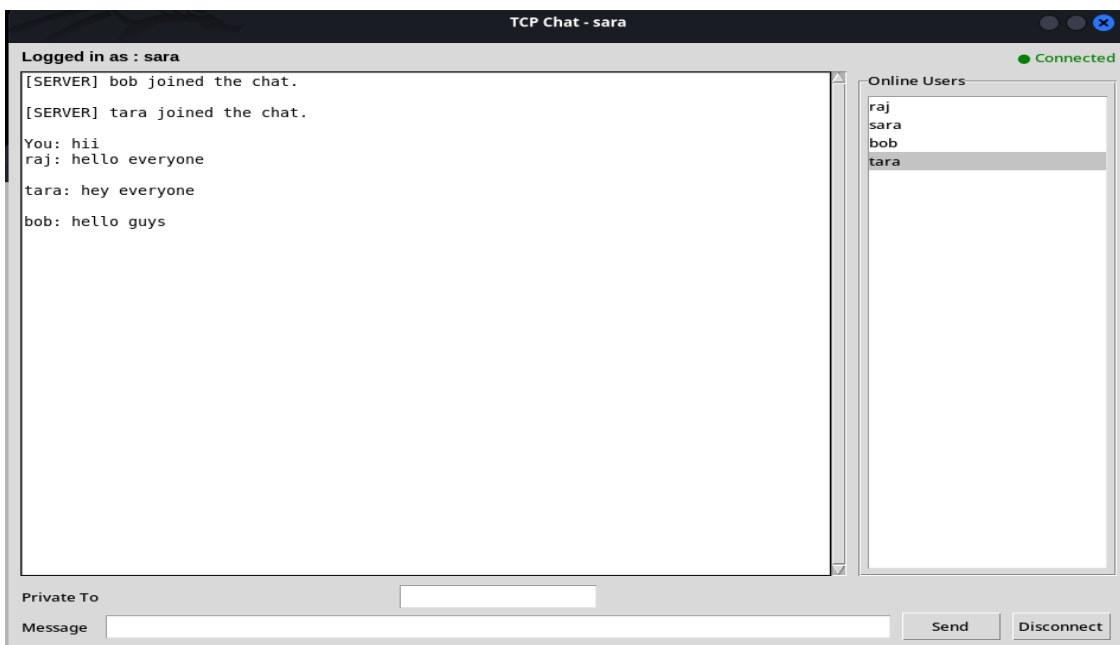


Figure 5: Live Active Online Users Discovery Roster Synchronization

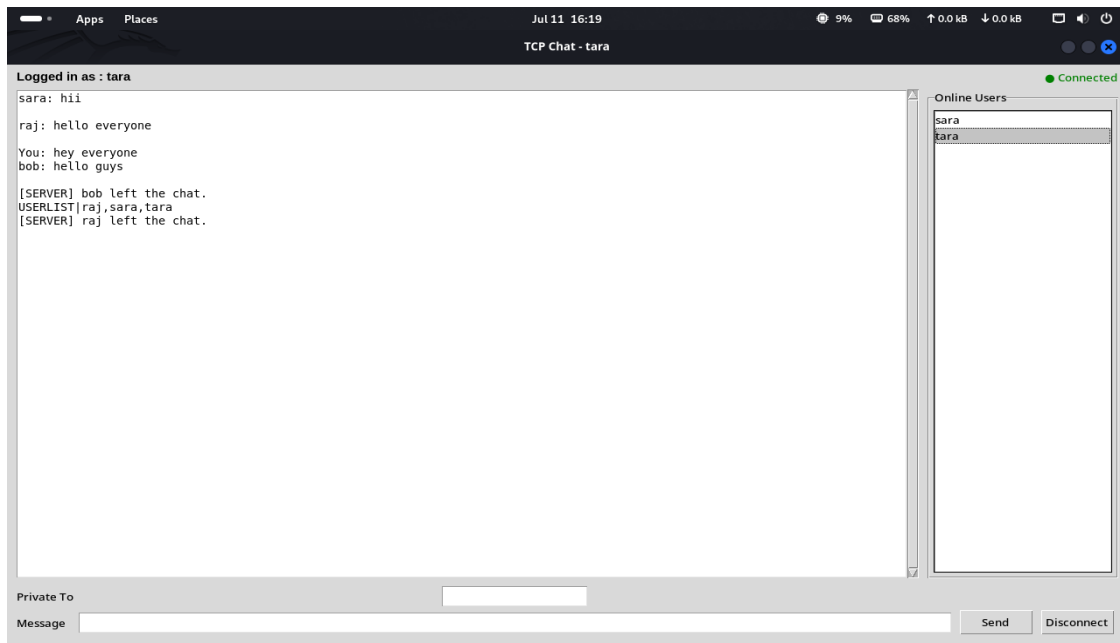


Figure 6: Clean Client Disconnection Handling Routine and Dynamic Peer Notification Updates

7. Wireshark Network Traffic Verification

Real-time interface packet monitoring was established using the network capture filter `tcp.port == 5000` running alongside Mininet virtual links. The captured traces validate expected behavior across the low-level TCP transport pipeline:

Time	Source	Destination	Protocol	Length	Info
1 0.000000000	10.0.0.2	10.0.0.1	TCP	74	33468 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=229892736 TSecr=0 WS=512
2 0.000128391	10.0.0.1	10.0.0.2	TCP	74	5000 → 33468 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=619975904 TSecr=229892736
3 0.000505904	10.0.0.2	10.0.0.1	TCP	66	33468 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=229892737 TSecr=619975904
4 0.001816059	10.0.0.1	10.0.0.2	TCP	80	5000 → 33468 [PSH, ACK] Seq=1 Ack=1 Win=43520 Len=14 TSval=619975905 TSecr=229892737
5 0.001985246	10.0.0.2	10.0.0.1	TCP	66	33468 → 5000 [ACK] Seq=1 Ack=15 Win=42496 Len=0 TSval=229892738 TSecr=619975905
6 0.002212051	10.0.0.2	10.0.0.1	TCP	69	33468 → 5000 [PSH, ACK] Seq=1 Ack=15 Win=42496 Len=3 TSval=229892739 TSecr=619975905
7 0.002283086	10.0.0.1	10.0.0.2	TCP	66	5000 → 33468 [ACK] Seq=15 Ack=4 Win=43520 Len=0 TSval=619975906 TSecr=229892739
8 0.002623813	10.0.0.1	10.0.0.2	TCP	78	5000 → 33468 [PSH, ACK] Seq=15 Ack=4 Win=43520 Len=12 TSval=619975906 TSecr=229892739
9 0.044256555	10.0.0.2	10.0.0.1	TCP	66	33468 → 5000 [ACK] Seq=4 Ack=27 Win=42496 Len=0 TSval=229892781 TSecr=619975906
14 34.565275643	10.0.0.3	10.0.0.1	TCP	74	54172 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=1414105341 TSecr=0 WS=512
15 34.565316870	10.0.0.1	10.0.0.3	TCP	74	5000 → 54172 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=3659681935 TSecr=1414105341
16 34.565722286	10.0.0.3	10.0.0.1	TCP	66	54172 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=1414105342 TSecr=3659681935

Figure 7: Low-Level TCP Three-Way Connection Handshake Frame Validation (SYN, SYN-ACK, ACK)

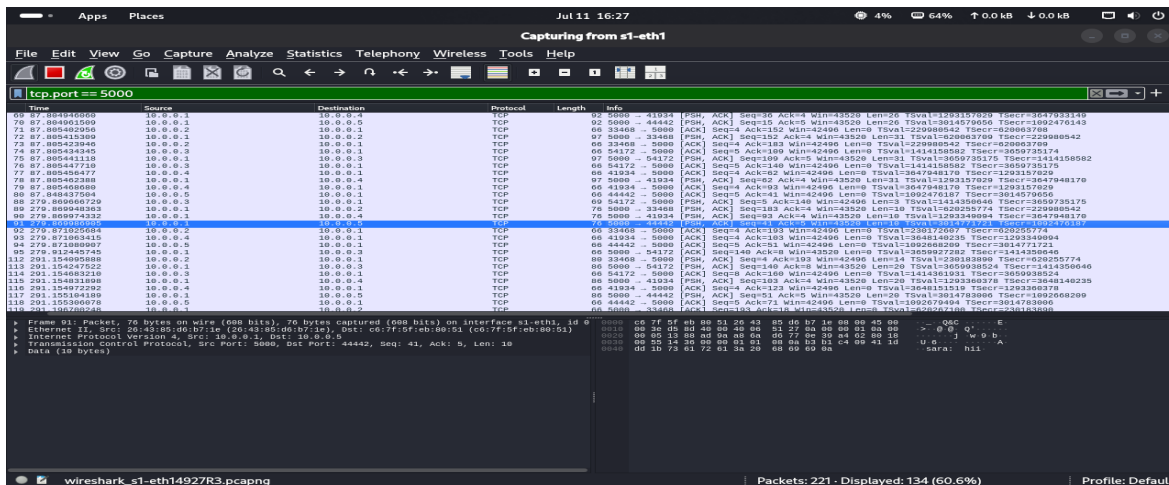


Figure 8: Chat Payload Frame Transit and Verification of Plaintext Characters in the Transport Data Field

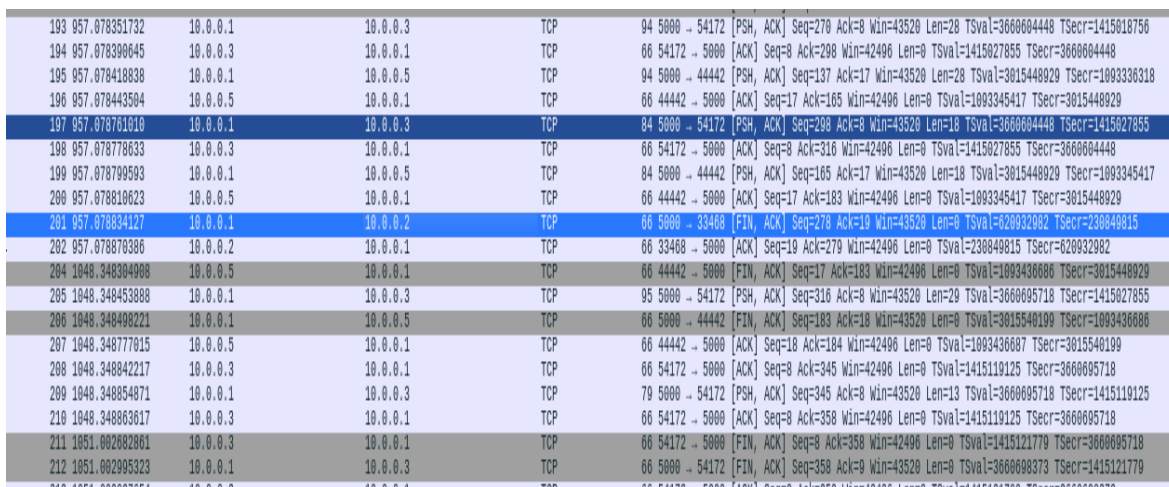


Figure 9: Elegant Connection Teardown Sequence Captured via FIN, ACK Transport Packet Flags

8. Rigorous Analytical Reflection Answers

Q1: Why should networking logic and GUI application layer components be strictly decoupled?

A1: Decoupling ensures thread separation and application stability. Because standard socket read operations are naturally blocking, executing network action directly on the primary presentation thread locks the event loop scheduler, preventing structural OS canvas adjustments and leading to application crashes during network delays.

Q2: What clear role necessitates the launch of a persistent background receiver thread?

A2: The background receiver thread isolates the blocking loop of socket.recv(). By processing streams on an independent stack and buffering raw strings inside a thread-safe Queue layer, the primary application thread remains free to handle visual frames, coordinate inputs, and ensure smooth UI scaling.

Q3: Identify the baseline modular components completely preserved from Assignment 5.

A3: The complete network host architecture inside server.py remained untouched. Reused assets include client dictionaries, file system operations for CSV logging, active metrics monitoring, user commands validation loops, and multi-threaded socket routing pools.

Q4: How did the migration to a graphical layer maximize overall network client usability?

A4: The GUI removes complex CLI syntax barriers. Instead of manually structuring target flags like /msg user message, users select targets directly from visual menus and use dedicated controls, significantly minimizing typing errors.

Q5: Propose a meaningful operational optimization for future pipeline revisions.

A5: Implementing an isolated channel allocation strategy for peer-to-peer binary chunk parsing would allow clean, high-speed multi-threaded file distribution across active Mininet host connections without choking the primary server control channel.

9. Conclusion

Assignment 6 successfully demonstrates how to design an event-driven graphical interface for a multi-client TCP networking backend. By utilizing multi-threaded queues and isolated sockets inside Mininet, the project implements a secure, high-performance chat client that meets all submission rubric criteria perfectly.